2026 Batch - Python FastAPI & AI Agents Standards - GenWorx Center of Excellence (COE) Developer Guide

Version: 1.0.2 **Status:** ACTIVE **Target Stack:** Python 3.11+, FastAPI, Pydantic v2, LangGraph, Celery, Faust, gRPC

---

1. Overview

This document provides a comprehensive template and guidelines for creating Python projects that adhere to GenWorx Center of Excellence (COE) standards. It covers both traditional Python applications and AI-powered projects with modular, scalable architectures.

Supported Project Types

- **Standard Python Projects**: REST APIs, CLI tools, data processing pipelines.
- **gRPC Services**: High-performance internal microservices.

Core Philosophy

- **Modularity**: Clear separation of concerns with well-defined module boundaries.
- **Scalability**: Architecture supports horizontal and vertical scaling.
- **Maintainability**: Code is clean, documented, and follows consistent patterns.
- **Security**: Security-first approach with secrets management and vulnerability scanning.
- **Observability**: Built-in logging, metrics, and tracing capabilities.
- **Async-First**: All I/O-bound operations must use async/await.

2. Project Structure Templates

We support two primary project structures: **Standard** (Rest API/Backend) and **gRPC Service**.

2.1 Standard Python Project (rest)

h for orchestration.

```
base-rest-service-template/
├── src/
│   ├── api/
│   │   ├── rest/                    # FastAPI application
│   │   │   ├── app.py              # FastAPI app setup
│   │   │   ├── routes/             # API routes
│   │   │   │   ├── health.py       # Health check endpoints
│   │   │   │   ├── sse.py          # SSE endpoints
│   │   │   │   └── websocket.py    # WebSocket endpoints
│   │   │   └── dependencies.py     # Dependency injection
│   │   └── middleware/             # FastAPI middleware
│   │       ├── cors.py
│   │       ├── logging.py
│   │       ├── error_handler.py
│   │       └── metrics.py
│   │
│   ├── core/                       # Core business logic
│   │   ├── exceptions/             # Exception hierarchy
│   │   └── services/               # Business logic services
│   │
│   ├── data/                       # Data access layer
│   │   ├── clients/                # Database clients
│   │   ├── repositories/           # Repository pattern
│   │   ├── models/                 # Data models
│   │   │   ├── postgres/           # SQLAlchemy models
│   │   │   └── mongo/              # MongoDB models
│   │   └── migrations/             # Database migrations
│   │
│   ├── handlers/                   # External service handlers
│   │   ├── grpc_clients/           # gRPC client handlers
│   │   └── http_clients/           # HTTP client handlers
│   │
│   ├── control/                    # Control layer
│   │   └── agents/                 # Intelligent agents
│   │
│   ├── observability/              # Observability stack
│   │   ├── logging/                # Structured logging
```

```
│   │   ├── metrics/                # Prometheus metrics
│   │   └── tracing/                # Distributed tracing
│   │
│   ├── schemas/                    # Pydantic schemas
│   ├── config/                     # Configuration
│   ├── constants/                  # Constants
│   ├── utils/                      # Utilities
│   └── main.py                     # Application entry point
│
├── examples/                       # Example implementations
│   ├── sse_client.py              # SSE consumer example
│   └── websocket_client.py        # WebSocket client example
│
├── docs/                           # Documentation
├── requirements/                   # Python dependencies
├── docker-compose.yml              # Docker services
├── Dockerfile                      # Application container
├── Makefile                        # Development commands
└── .env.example                    # Example environment variables
```

2.3 gRPC Service Project

```
base-grpc-service-template/
├── src/                          # Source code
│   ├── api/                      # API layer
│   │   ├── grpc/                 # gRPC server and services
│   │   │   ├── services/         # Service implementations
│   │   │   │   ├── health_service.py
│   │   │   │   └── streaming_service.py
│   │   │   └── server.py         # gRPC server setup
│   │   ├── interceptors/         # gRPC interceptors
│   │   │   ├── auth_interceptor.py
│   │   │   ├── error_interceptor.py
│   │   │   ├── logging_interceptor.py
│   │   │   └── metrics_interceptor.py
│   │   └── dependencies/         # Dependency injection
│   │
│   ├── core/                     # Core business logic
│   │   ├── exceptions/           # Exception hierarchy
│   │   │   ├── base.py           # Base exception classes
│   │   │   └── mapper.py         # gRPC status code mapping
│   │   └── services/             # Business logic services
│   │
│   ├── data/                     # Data access layer
│   │   ├── clients/              # Database clients
│   │   │   ├── postgres_client.py
│   │   │   ├── mongo_client.py
│   │   │   └── redis_client.py
│   │   ├── repositories/         # Repository pattern
│   │   │   ├── base_postgres_repository.py
│   │   │   └── base_mongo_repository.py
│   │   ├── models/               # Data models
│   │   │   ├── postgres/         # SQLAlchemy models
│   │   │   └── mongo/            # MongoDB models
│   │   └── migrations/           # Database migrations
│   │
│   ├── handlers/                 # External service handlers
│   │   ├── grpc_clients/         # gRPC client handlers
│   │   │   └── base_grpc_client.py
│   │   └── http_clients/         # HTTP client handlers
│   │       └── base_http_client.py
│   │
│   ├── control/                  # Control layer
│   │   └── agents/               # Intelligent agents
│   │
│   ├── observability/            # Observability stack
│   │   ├── logging/              # Structured logging
│   │   │   ├── logger.py
│   │   │   └── formatters.py
│   │   ├── metrics/              # Prometheus metrics
│   │   │   └── collectors.py
│   │   └── tracing/              # Distributed tracing
│   │
│   ├── schemas/                  # Pydantic schemas
│   ├── config/                   # Configuration
│   │   └── settings.py           # Pydantic settings
│   ├── constants/                # Application constants
│   ├── utils/                    # Utilities
│   │   ├── context.py            # Context management
│   │   └── helpers/              # Helper functions
│   └── main.py                   # Application entry point
│
├── examples/                     # Example implementations
│   └── streaming_clients.py      # gRPC streaming examples
│
├── scripts/                      # Helper scripts
├── docs/                         # Documentation
```

```
│   ├── development.md            # Developer guide
│   ├── proto_library.md          # Proto library pattern
│   └── api/                      # API documentation
│
├── requirements/                 # Python dependencies
│   ├── base.txt                  # Production dependencies
│   ├── dev.txt                   # Development dependencies
│   └── test.txt                  # Testing dependencies
│
├── docker-compose.yml            # Docker services
├── Dockerfile                    # Application container
├── Makefile                      # Development commands
├── pyproject.toml                # Poetry/project configuration
├── pytest.ini                    # Pytest configuration
├── ruff.toml                     # Ruff linter configuration
├── .env.example                  # Example environment variables
├── .gitignore                    # Git ignore patterns
└── .pre-commit-config.yaml       # Pre-commit hooks
```

3. Environment Setup

3.1 Prerequisites

- **Python**: 3.11+
- **Package Manager**: uv (Required for speed and determinism)
- **Container Runtime**: Docker Desktop 4.20+
- **Message Broker**: Redis (for Celery) / Kafka (for Faust)

3.2 Installation (Using uv)

```
# 1. Install uv
curl -LsSf https://astral.sh/uv/install.sh | sh

# 2. Initialize Project
uv init my-project
cd my-project

# 3. Create Virtual Environment
uv venv
source .venv/bin/activate

# 4. Install Dependencies
uv pip install -e ".[dev]"
```

3.3 Configuration

We follow the **12-Factor App** methodology. All configuration must be environment-variable driven.

1. Copy .env.template to .env.
2. Never commit .env to Git.

Sample .env.template:

```
APP_ENV=development
LOG_LEVEL=INFO
DATABASE_URL=postgresql://user:pass@localhost:5432/dbname

# Celery (Redis)
CELERY_BROKER_URL=redis://localhost:6379/0
CELERY_RESULT_BACKEND=redis://localhost:6379/1

# Kafka (Faust)
KAFKA_BROKER_URL=kafka://localhost:9092
KAFKA_TOPIC_PREFIX=genworx-

OPENAI_API_KEY=sk-...
```

4. COE Standards & Best Practices

4.1 Code Style (Ruff)

We utilize **Ruff** as a single tool to replace Black, Flake8, Isort, and PyUpgrade.

Rules: - **Line Length**: 88 characters. - **Imports**: Sorted and grouped automatically. - **Formatting**: Double quotes, trailing commas.

pyproject.toml Configuration:

```
[tool.ruff]
line-length = 88
target-version = "py311"

[tool.ruff.lint]
select = ["E", "W", "F", "I", "B", "UP", "N", "SIM", "C4"]
ignore = []

[tool.ruff.format]
quote-style = "double"
indent-style = "space
```

4.2 Type Checking (MyPy)

Strict static type checking is **mandatory**.

```
[tool.mypy]
python_version = "3.11"
strict = true
disallow_untyped_defs = true
no_implicit_optional = true
```

4.3 Dependency Management

Dependencies are defined in pyproject.toml and locked with uv.lock.

```
[project]
name = "my-project"
version = "0.1.0"
dependencies = [
    "fastapi>=0.109.0",
    "pydantic>=2.0.0",
    "sqlalchemy>=2.0.0",
    "celery>=5.3.0",
    "redis>=5.0.0",
    "faust-streaming>=0.10.0",
    "grpcio>=1.59.0",
    "grpcio-tools>=1.59.0"
]

[project.optional-dependencies]
dev = [
    "pytest",
    "ruff",
    "mypy"
]
ai = [
    "langchain",
    "langgraph",
    "openai"
]
```

4.4 Git Workflow

- **Main Branch**: main (Production).
- **Format**: Conventional Commits (feat:, fix:, docs:, chore:).
- **Pre-commit Hooks**: Must run ruff check, ruff format, and mypy before commit.

5. Development Workflows

5.1 Local Development

```
# Run with Docker Compose (Recommended)
docker-compose up --build
```

5.2 Database Migrations (Alembic)

```
# Create migration
alembic revision --autogenerate -m "Add users table"

# Apply migration
alembic upgrade head
```

---

6. Security Guidelines

6.1 Secrets

- **NEVER** commit secrets.
- Use pydantic-settings to read from environment.

6.2 API Security

- **Auth**: OAuth2 / OIDC with JWT.
- **Validation**: Strict Pydantic models for all inputs.
- **Rate Limiting**: Use slowapi to prevent abuse.

6.3 Docker Security

- Run as non-root user (USER appuser).
- Use multi-stage builds to minimize image size.
- Scan images regularly with Trivy.

7. Documentation Standards

7.1 Code

- **Docstrings**: Google Style docstrings for all public APIs.
- **Type Hints**: Mandatory for self-documentation.

7.2 API

- **OpenAPI**: Auto-generated by FastAPI.
- **Descriptions**: Add summary and description to all routes.

8. Troubleshooting

Common Issues

- **Module Not Found**: Ensure uv pip install -e . was run.
- **DB Connection**: Check Docker container status (docker ps).
- **LLM Rate Limits**: Implement exponential backoff using tenacity.

Version: 1.0.3
Last Updated: 2026-01-19
Maintained By: GenWorx COE Team